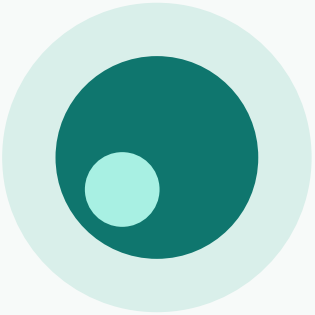
MEDs

Recordatorios de medicación, sin ruido

Documentación funcional y técnica

Guía de uso, arquitectura MVVM y cobertura de la rúbrica del proyecto integrador de aplicación móvil.



Kotlin · Jetpack Compose

MVVM · StateFlow

Firebase Auth + Firestore

Versión 1.0 · versionCode 1
Noviembre 2026

Documento de trabajo del equipo
com.meds.app

Contenido

1. Resumen ejecutivo . . . 3

2. Guía de uso: recorrido completo . . . 3

3. Arquitectura MVVM y organización del código . . . 6

4. Consumo de API y tratamiento de datos . . . 7

5. Interfaz con Jetpack Compose y navegación . . . 8

6. Gestión de datos y persistencia local . . . 9

7. Integraciones del dispositivo y generación de archivos . . . 10

8. Estabilidad, validaciones y experiencia de uso . . . 11

9. Repositorio, documentación y versión entregable . . . 12

10. Demostración práctica y defensa técnica . . . 13

11. Cobertura de la rúbrica . . . 14

12. Anexos . . . 14

1. Resumen ejecutivo

MEDs es una aplicación Android nativa para recordar medicamentos y sostener la constancia diaria. Está pensada para personas con TDAH y para quienes las acompañan: reduce la carga cognitiva, muestra solo lo necesario y convierte cada toma en una acción de un toque.

El proyecto no se limita a listar pastillas. Modela dos roles (paciente y acompañante), un jardín que crece con la constancia, recordatorios locales, un widget de próxima toma y un mecanismo de vinculación por QR para que una persona de confianza pueda ver el día del paciente y enviarle un recordatorio.

La app usa Kotlin, Jetpack Compose y MVVM; Firebase Authentication y Cloud Firestore como backend en tiempo real; DataStore para preferencias locales; y tres integraciones del dispositivo: QR, biometría y widget.

1.1 Problema y propuesta

- Olvidar una toma no es falta de voluntad: es carga cognitiva. Las apps genéricas añaden pasos, notificaciones ruidosas y pantallas densas.
- MEDs reduce la fricción: una toma se confirma con un toque; posponer cuesta otro toque; el estado del día se entiende de un vistazo.
- El modo acompañante resuelve el aislamiento: alguien de confianza ve el día y puede enviar un recordatorio, sin acceso al historial completo.
- La constancia se refuerza con una planta y una racha, no con castigos ni culpa.

1.2 Ficha técnica

Aspecto	Detalle
Nombre y paquete	MEDs · com.meds.app
Propósito	Recordatorios de medicación y acompañamiento para personas con TDAH
Plataforma	Android 8.0 o superior (minSdk 26, targetSdk 37)
Lenguaje e interfaz	Kotlin, Jetpack Compose, Material 3
Arquitectura	MVVM + Repository, StateFlow y corrutinas; inyección manual
Backend	Firebase Authentication + Cloud Firestore (caché offline)
Persistencia local	DataStore Preferences, SharedPreferences del widget y AlarmManager
Integraciones	QR con ZXing, biometría con androidx.biometric y widget AppWidgetProvider
Versión	1.0 · versionCode 1

> Para la defensa
> Si hay que resumir MEDs en una frase: es un sistema de adherencia con dos roles, no un simple listado de medicamentos. La pantalla Hoy responde tres preguntas: qué toca, a qué hora y qué hice.

2. Guía de uso: recorrido completo

2.1 Instalar y abrir

La variante de depuración se compila con `.\gradlew.bat :app:assembleDebug` y queda en `app\build\outputs\apk\debug\app-debug.apk`. Se instala con `adb install -r app-debug.apk`. El proyecto necesita un proyecto Firebase con los proveedores Anónimo y Correo/Contraseña habilitados, además de las reglas de `firestore.rules` publicadas.

2.2 Cuenta y acceso

Al abrir por primera vez, la app pide autenticarse. Hay tres caminos:

- Crear cuenta con correo y contraseña (mínimo 6 caracteres).
- Iniciar sesión si la cuenta ya existe, con recuperación de contraseña por correo.
- Continuar sin cuenta: se crea una sesión anónima y se muestra una advertencia con los riesgos de no vincular un correo.

La cuenta anónima es usable por completo. Desde Ajustes, Cuenta se puede vincular un correo; la vinculación conserva el mismo `uid` y, por tanto, los medicamentos, tomas e historial. Si se cierra sesión sin vincular, la cuenta anónima no se puede recuperar. El registro y el login restauran el rol y el nombre desde Firestore, de modo que el usuario recupera su espacio en otro dispositivo.

2.3 Onboarding

- Se elige el rol: paciente o acompañante.
- Se escribe el nombre y se completa la introducción.
- Se solicita permiso de notificaciones y, si el sistema lo permite, de alarmas exactas.
- La app explica que el modo acompañante usa un código temporal de 10 minutos.

2.4 Paciente

- Hoy: próxima toma, estado del día, botones Ya la tomé y Posponer 10 min, acceso al historial y al QR de acompañante.
- Medicamentos: lista, alta y edición; cada medicamento puede tener una o varias horas.
- Jardín: planta, nivel, XP, racha y total de tomas.
- Ajustes: cuenta, vinculación de correo, cierre de sesión y versión.

Al agregar un medicamento se guardan el medicamento y su horario en un batch; se programan alarmas locales y se generan las tomas del día.

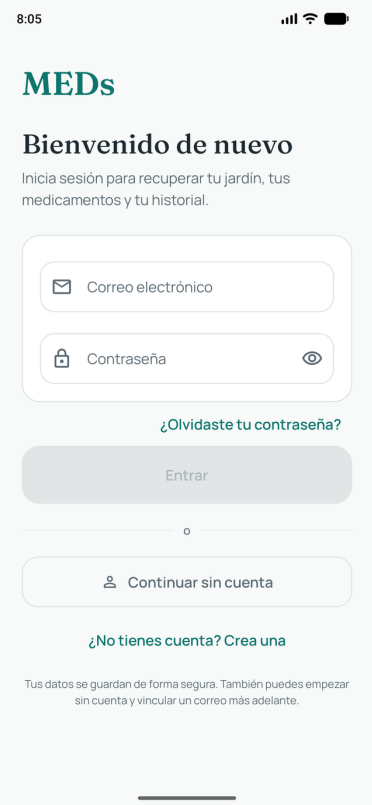
2.5 Acompañante

- Escanear el QR del paciente o escribir el código de 8 caracteres.
- Ver en tiempo real las tomas del día (no el historial completo).
- Enviar un recordatorio (nudge) con un mensaje breve.
- Desvincularse cuando quiera; el paciente también puede retirar el acceso.

El código dura 10 minutos y es de un solo uso. El paciente puede generar uno nuevo cuando quiera desde la pantalla Acompañante.

2.6 Widget, alarmas y biometría

- Widget: muestra la próxima toma pendiente; se actualiza cuando cambian las tomas y cada 30 minutos.
- Alarmas: notificaciones locales con acciones Ya la tomé y Posponer 10 min; se reprograman al reiniciar el dispositivo.
- Biometría: si el dispositivo tiene huella o rostro inscrito, la app pide desbloqueo antes de mostrar datos; si no hay biometría, no bloquea.



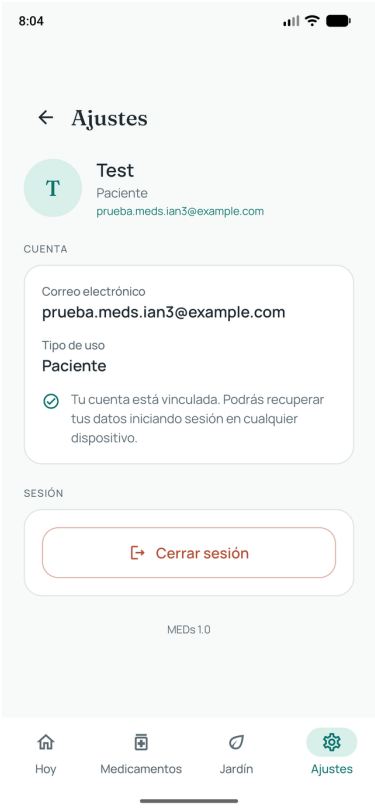
Inicio de sesión con correo y contraseña



Pantalla Hoy con la próxima toma



Alta de medicamento y horario



Ajustes: cuenta vinculada y cierre de sesión

Capturas tomadas en emulador Pixel 6 Pro y Xiaomi Redmi; la versión instalada puede variar en detalles visuales.

3. Arquitectura MVVM y organización del código

La app separa interfaz, estado, acceso a datos y fuentes. Ninguna pantalla habla directamente con Firebase: las pantallas observan un StateFlow del ViewModel y envían eventos; el ViewModel llama al Repository; el Repository encapsula el SDK y las reglas de negocio.

```
Compose (pantallas)
    |  eventos: onClick, onValueChange
    v
ViewModel (StateFlow + corrutinas)
    |  llamadas suspend
    v
Repository (reglas de negocio + mapeo)
    |  SDK / DataStore / AlarmManager
    v
Firebase Auth, Cloud Firestore, DataStore, alarmas
    |
    +--> Flows reactivos --> ViewModel --> StateFlow --> Compose
```

3.1 Capas y responsabilidades

Capa	Responsabilidad	Ejemplos
Interfaz (Compose)	Dibuja el estado y emite eventos; no accede a datos.	ui/screens, ui/components, MedsRoot
Estado (ViewModel)	Expone StateFlow, lanza corrutinas y traduce errores.	AppViewModel, PatientViewModel, CompanionViewModel
Datos (Repository)	Operaciones, batches, mapeo, reglas de negocio y Result.	AuthRepository, PatientRepository, CompanionRepository
Fuentes	Identidad, base de datos, preferencias y alarmas.	FirebaseAuth, FirebaseFirestore, PreferencesStore, AlarmScheduler

3.2 Organización de paquetes

Paquete	Contenido
data/model	Modelos inmutables: Medication, Schedule, DoseLog, PlantState, Nudge, PairingInvite.
data/repository	AuthRepository, UserRepository, PatientRepository, CompanionRepository, FirestoreMappers.
data/local	PreferencesStore con DataStore: rol, nombre y onboarding.
ui/screens/auth	AuthScreen y AnonymousWarningScreen.
ui/screens/settings	AccountSettingsScreen: correo, vinculación y cierre de sesión.
ui/screens/patient	PatientApp, HomeScreen, MedicationsScreen, MedicationEditorScreen, HistoryScreen, GardenScreen y PatientCompanionScreen.
ui/screens/companion	CompanionApp y CompanionViewModel.
ui/screens/shared	GateScreens (carga y error) y BiometricGate.
ui/components, ui/theme	Sistema de diseño: tarjetas, botones, dosis, tipografía y color.
notifications, widget, util	Alarmas, notificaciones, widget de próxima toma, fechas, QR y códigos.

3.3 Estado, corrutinas y errores

Los ViewModels exponen un único StateFlow. Para combinar fuentes se usa `combine`; para cambiar de día o de paciente, `flatMapLatest`; para evitar trabajo innecesario, `stateIn(WhileSubscribed(5_000))`. Las pantallas recogen el estado con `collectAsStateWithLifecycle`.

```
val state: StateFlow<PatientUiState> = combine(coreFlow, extras) { core, extra ->
    core.copy(error = extra.error, saving = extra.saving)
}.stateIn(
    scope = viewModelScope,
    started = SharingStarted.WhileSubscribed(5_000),
    initialValue = PatientUiState()
)
```

Las escrituras son suspend y devuelven `Result`; los errores se traducen a mensajes comprensibles y se muestran con snackbar. Los ViewModels no bloquean el hilo principal. AppViewModel distingue autenticación (userId, email, isAnonymous) de perfil (rol, nombre, onboarding); la restauración de perfil tiene timeout de 8 s y se cancela si cambia el usuario, para evitar condiciones de carrera.

3.4 Inyección de dependencias

La inyección es manual y explícita. Los ViewModels construyen sus repositorios (o los reciben con valores por defecto) y `MedsApp` inicializa Firebase y las alarmas. Es una decisión deliberada: la app es pequeña, las dependencias son pocas y visibles, y no se introduce Hilt para no añadir complejidad. Los repositorios aceptan `FirebaseFirestore` como parámetro, lo que permite sustituirlos en pruebas.

3.5 Dos recorridos de extremo a extremo

Recorrido	Cadena completa
Inicio de sesión	AuthScreen -> AppViewModel.login -> AuthRepository.login -> FirebaseAuth -> authStateFlow -> UserRepository.loadProfile -> PreferencesStore.restoreProfile -> MedsRoot.
Marcar toma	HomeScreen -> PatientViewModel.takeDose -> PatientRepository.takeDose -> batch (doseLog + plant + stock) -> AlarmScheduler.cancel -> Firestore -> flows -> UI.
Vincular correo	AccountSettingsScreen -> AppViewModel.linkEmail -> AuthRepository.linkAnonymousToEmail -> FirebaseAuth.linkWithCredential -> actualización explícita del estado (el uid no cambia, por eso no se espera al listener).

Nivel sobresaliente del criterio 1
Pantallas que observan ViewModel; Repository que encapsula datos; StateFlow, corrutinas e inyección explícita. Es exactamente el descriptor del nivel 4: separación de responsabilidades y manejo observable del estado.

4. Consumo de API y tratamiento de datos

El servicio externo es Firebase: Authentication para identidad y Cloud Firestore como base de datos documental en tiempo real. No hay datos simulados: las pantallas leen y escriben documentos reales del proyecto `medsproto`.

4.1 Operaciones principales

Operación	Técnica	Dónde
Registro, login, vínculo y recuperación	FirebaseAuth (correo/contraseña y anónima)	AuthRepository
Observar medicamentos, horarios, tomas y planta	Snapshots en tiempo real convertidos a Flow	PatientRepository
Guardar medicamento y horario	Batch de dos documentos	PatientRepository
Marcar toma	Batch: doseLog, planta y stock; cancela la alarma	PatientRepository
Invitar y vincular acompañante	Batch + validación de invitación en reglas	CompanionRepository
Enviar y leer nudges	set / update con permisos por rol	CompanionRepository
Perfil y rol	set(merge) en users/{uid}	UserRepository

4.2 Tiempo real, caché y estados

Las consultas se exponen como Flow mediante `callbackFlow` y listeners de snapshot. Firestore mantiene una caché local; si no hay red, la app muestra lo último conocido y se actualiza al reconectar. Cada pantalla tiene estados de carga, vacío, error y reintento.

Los errores se normalizan: credenciales inválidas, correo en uso, contraseña débil, red, demasiados intentos y permisos. En el modo acompañante, un `PERMISSION_DENIED` se traduce a 'El acceso a este paciente fue retirado'. La restauración de perfil usa `withTimeoutOrNull(8_000)` para no dejar la app bloqueada si Firestore tarda.

4.3 Seguridad de los datos

Las reglas de Firestore validan que cada usuario solo escriba sus propios documentos; el acompañante solo lee; las invitaciones son de un solo uso y expiran; los nudges solo los crea un acompañante vinculado. El cliente nunca decide los permisos.

```
users/{uid}                          perfil, rol y nombre
  medications/{medicationId}          medicamentos
  schedules/{scheduleId}              horas y dias
  doseLogs/{logId}                    tomas del dia e historial
  plant/state                         nivel, xp, racha
  companions/{companionUid}           vinculo (solo lectura para el acompanante)
  nudges/{nudgeId}                    recordatorios del acompanante
users/{companionUid}/linkedPatients/{patientUid}
pairingInvites/{code}                 codigo temporal, un solo uso
```

> Matiz importante para la defensa
> La rúbrica menciona Retrofit o un cliente equivalente. Firebase es un backend real y el SDK es su cliente; si la cátedra exige explícitamente REST/Retrofit, el equipo debe reforzar este punto con una API pública (por ejemplo, openFDA para información de medicamentos) y documentar el flujo. Hoy no está implementada: es el principal riesgo del criterio 2.

5. Interfaz con Jetpack Compose y navegación

Toda la interfaz es Compose + Material 3. El tema 'Sereno clínico' usa Manrope para texto y Fraunces para títulos, con paleta teal, fondos claros y modo oscuro. Los componentes reutilizables (MedsCard, PrimaryActionButton, DoseComponents) mantienen la consistencia.

5.1 Navegación

MedsRoot decide la pantalla raíz según el estado: carga, autenticación, advertencia anónima, selección de rol, onboarding o app. En modo paciente, un NavHost maneja las pestañas y pantallas internas; la barra inferior solo aparece en las rutas de nivel superior.

Ruta	Pantalla	Propósito
home	HomeScreen	Próxima toma, tomas del día y acceso a historial/QR.
medications	MedicationsScreen	Lista y edición de medicamentos.
garden	GardenScreen	Planta, nivel, XP y racha.
settings	AccountSettingsScreen	Cuenta, vinculación de correo y cierre de sesión.
history	HistoryScreen	Últimos 7 días de tomas.
companions	PatientCcompanionScreen	Invitación QR y personas vinculadas.
medication_editor/new	MedicationEditorScreen	Alta de medicamento.
medication_editor/{id}	MedicationEditorScreen	Edición de medicamento y horario.

El modo acompañante usa estado (`showSettings`, `selectedPatient`) en lugar de un NavHost: es un flujo más corto y evita una ruta innecesaria. El botón Atrás se maneja con BackHandler en la advertencia anónima, el registro y los ajustes del acompañante. La navegación conserva estado con `saveState/restoreState` y usa `launchSingleTop` para evitar duplicados.

5.2 Inventario de pantallas

Pantalla	Qué resuelve
AuthScreen	Login y registro con validación, visibilidad de contraseña y recuperación.
AnonymousWarningScreen	Explica los riesgos de usar sin cuenta y ofrece crear cuenta o continuar.
RoleScreen	Selección de paciente o acompañante.
OnboardingScreen	Nombre, introducción y permisos de notificaciones/alarmas.
HomeScreen	Estado del día, próxima toma y acciones de toma/posponer.
MedicationsScreen / MedicationEditorScreen	Gestión completa del medicamento y sus horas.
HistoryScreen / GardenScreen	Historial de 7 días y progreso lúdico.
PatientCompanionScreen	QR temporal y administración de acompañantes.
AccountSettingsScreen	Cuenta, vinculación de correo, cierre de sesión y versión.
CompanionApp	Vinculación por QR/código, panel del día, nudges y desvinculación.

5.3 Accesibilidad y adaptación

- Contenido desplazable y layouts que se adaptan al ancho; probado en 1080x2400 (Xiaomi) y 1440x3120 (emulador).
- Descripciones de contenido en iconos interactivos (Ajustes, volver, visibilidad de contraseña).
- Textos de error y ayudas junto a los campos; validación en el momento.
- Objetivos táctiles de al menos 48 dp en botones principales.
- Modo claro/oscuro según el sistema, con contraste cuidado.

6. Gestión de datos y persistencia local

Dato	Mecanismo	Supervivencia	Dónde
Rol, nombre y onboarding	DataStore Preferences	Reinicio de la app	PreferencesStore
Medicamentos, horarios, tomas, planta y vínculos	Cloud Firestore + caché offline	Reinicio y reconexión	Repositories
Próxima toma del widget	SharedPreferences	Reinicio y cambios de toma	NextDoseWidgetProvider
Alarmas locales	AlarmManager + SharedPreferences	Reinicio del dispositivo	AlarmScheduler / BootReceiver

6.1 Por qué DataStore y no Room

DataStore se usa para preferencias pequeñas y reactivas (rol, nombre, onboarding). Firestore es la fuente de verdad de los datos de dominio y su caché offline cubre la lectura sin red. No se usa Room porque duplicaría la fuente de verdad y añadiría sincronización; si en el futuro se necesita historial local completo o consultas complejas offline, Room sería la evolución natural.

El widget guarda un snapshot en SharedPreferences porque un AppWidgetProvider no puede observar Flows de Firestore de forma directa; la app actualiza ese snapshot cuando cambian las tomas. Las alarmas se programan con AlarmManager y se reprograman en BootReceiver; el sistema las conserva aunque la app se cierre.

6.2 Evidencia de recuperación

- Tras reiniciar la app, el rol y el nombre se restauran.
- Tras iniciar sesión en otro dispositivo, el perfil se recupera desde Firestore.
- El widget conserva la última próxima toma y avisa si el día cambió.

- Las alarmas se reprograman al reiniciar el dispositivo.

> Riesgo controlado
> Una sesión anónima sin correo no se puede recuperar tras desinstalar la app o cerrar sesión. La interfaz lo advierte antes de continuar y ofrece vincular un correo desde Ajustes; la vinculación conserva el mismo uid y todos los datos.

7. Integraciones del dispositivo y generación de archivos

La rúbrica pide tres funcionalidades entre PDF, QR, cámara/galería, biometría, audio o widget. MEDs implementa tres: QR (generar y escanear), biometría y widget. Además incluye alarmas y notificaciones locales, que no cuentan para las tres pero mejoran la experiencia.

7.1 QR para vincular al acompañante

El paciente genera una invitación con `QrCodeGenerator` (ZXing) y la muestra como QR con el contenido `meds://pair?code=XXXXXXXX`. El acompañante puede escanearlo con ZXing Embedded o escribir el código de 8 caracteres. La invitación dura 10 minutos, es de un solo uso y las reglas validan que el paciente sea el dueño. El acompañante solo ve las tomas del día, no el historial completo.

7.2 Biometría

BiometricGate usa `BiometricPrompt` con `BIOMETRIC_WEAK`. Si el dispositivo tiene huella o rostro inscrito, la app pide desbloqueo al abrir; si no hay biometría disponible, no bloquea y deja pasar. El desbloqueo se conserva mientras la actividad vive; al reiniciar el proceso se vuelve a pedir.

7.3 Widget de próxima toma

NextDoseWidgetProvider es un AppWidgetProvider con RemoteViews. Guarda un snapshot de la próxima toma pendiente o pospuesta y lo pinta en el launcher. Se actualiza cuando cambian las tomas y cada 30 minutos (`updatePeriodMillis=1800000`). Si el día cambió, invita a abrir MEDs para refrescar.

7.4 Alarmas y notificaciones (extra)

AlarmScheduler programa alarmas exactas cuando el sistema lo permite y usa alarmas inexactas como respaldo. Las notificaciones incluyen las acciones Ya la tomé y Posponer 10 min, que ejecutan DoseActionReceiver. BootReceiver reprograma todo al reiniciar el dispositivo.

7.5 Permisos declarados

Permiso	Uso
INTERNET	Firebase Authentication y Cloud Firestore.
POST_NOTIFICATIONS	Recordatorios locales de cada toma (Android 13+).
SCHEDULE_EXACT_ALARM	Alarmas exactas; si no se concede, se usan inexactas.
RECEIVE_BOOT_COMPLETED	Reprogramar alarmas tras reiniciar.
CAMERA	Escaneo del QR de vinculación con ZXing.
VIBRATE / WAKE_LOCK	Notificaciones y alarmas confiables.



QR temporal de acompañante



Bloqueo biométrico



Widget de próxima toma

No se declaran permisos sin uso: cada permiso del manifiesto tiene una función concreta en la app.

8. Estabilidad, validaciones y experiencia de uso

8.1 Validaciones

Campo / flujo	Validación
Correo	Formato con expresión regular; error junto al campo.
Contraseña	Mínimo 6 caracteres; confirmación obligatoria en registro y vinculación.
Medicamento	Nombre y al menos una hora; dosis y forma con valores por defecto.
Código de acompañante	8 caracteres alfanuméricos; existencia, expiración y no uso previo.
Nudge	Mensaje obligatorio, máximo 160 caracteres.
Stock	Se descuenta solo si hay existencias; nunca queda negativo.

8.2 Errores y recuperación

Situación	Respuesta
Credenciales inválidas	Mensaje claro; no se cierra la app.
Correo ya registrado	Se sugiere iniciar sesión o usar otro correo al vincular.
Contraseña débil	Se indica el mínimo de 6 caracteres.
Sin conexión	Firestore usa caché; se muestra el último estado y se reintenta.
Permiso retirado al acompañante	Mensaje específico y opción de desvincularse.
Perfil lento	Timeout de 8 s; la app no queda bloqueada.
Doble toque en una toma	Se comprueba el estado antes de volver a escribir.

8.3 Salvaguardas de experiencia

- Advertencia antes de continuar sin cuenta y confirmación al cerrar sesión.
- Estados de carga, vacío y error; snackbars para fallos de escritura.
- Cambio de día detectado cada 30 segundos; el widget avisa si quedó obsoleto.

- La confirmación de toma es idempotente y cancela la notificación correspondiente.
- El modo acompañante muestra solo el día y permite desvincularse.

8.4 Pruebas realizadas

Se verificaron los recorridos en emulador Pixel 6 Pro y en un Xiaomi Redmi con Android 15: login, registro, modo anónimo, advertencia, vinculación de correo, cierre de sesión, restauración de perfil, alta de medicamento, toma, historial, QR, biometría y widget. Durante las pruebas se corrigieron dos defectos reales: la UI no se actualizaba al vincular el correo (Firebase no dispara el listener porque el uid no cambia) y la restauración de perfil podía reescribir preferencias del usuario anterior después de cerrar sesión.

> Limitaciones conocidas
> Una cuenta anónima sin correo se pierde al desinstalar o cerrar sesión; Firestore Spark no envía push con la app totalmente cerrada (las alarmas locales sí funcionan); el historial del acompañante se limita al día actual; el repositorio Git y la firma de release están pendientes (sección 9).

9. Repositorio, documentación y versión entregable

9.1 Estado actual

Elemento	Estado
Código fuente	Proyecto Android complete en Kotlin/Compose.
README	Actualizado con stack, Firebase, compilación, flujo de autenticación y estructura.
APK	app\build\outputs\apk\debug\app-debug.apk (debug, versión 1.0).
Evidencias	Carpeta evidencias/ con 19 capturas: rol, onboarding, home, editor, widget, biometría, login, ajustes y QR.
Versión	versionName 1.0, versionCode 1.
Repositorio Git	Pendiente de inicializar y subir con ramas main/develop/feature.
Firma de release	Pendiente: configurar keystore y generar APK/AAB firmado.

9.2 Acciones para llegar al nivel sobresaliente

- Inicializar el repositorio con `main`, `develop` y ramas `feature`; historial de commits comprensible.
- Generar un release firmado y conservar el APK junto al código.
- Mantener las evidencias actualizadas (login, ajustes y QR ya están en la carpeta).
- Publicar las reglas de Firestore y verificar escrituras en producción.
- Añadir una política de privacidad y el borrado de cuenta para Play Store (fuera de la rúbrica).

> Checklist antes de entregar
> Reglas publicadas · README al día · APK instalable · evidencias · repo Git · release firmado · cuenta de demo con correo vinculado · guion de demostración ensayado.

10. Demostración práctica y defensa técnica

10.1 Guion de 10 minutos

Minuto	Qué mostrar
0:00 - 1:00	Problema y propuesta: adherencia, TDAH, dos roles y promesa de bajo ruido.
1:00 - 2:30	Login o registro; advertencia anónima; vincular correo en Ajustes y mostrar que los datos se conservan.
2:30 - 4:00	Onboarding, alta de medicamento y pantalla Hoy; marcar una toma y posponer otra.
4:00 - 5:30	Historial y jardín; explicar XP, racha y planta.
5:30 - 7:30	Modo acompañante en un segundo dispositivo: generar QR, escanear, ver el día en tiempo real y enviar un nudge.
7:30 - 8:30	Widget, notificación con acciones y desbloqueo biométrico.
8:30 - 10:00	Arquitectura: abrir Repository, ViewModel y UI; explicar API -> Repository -> ViewModel -> UI, reglas y persistencia.

10.2 Preguntas probables

Pregunta	Respuesta corta
¿Dónde está la llamada a la API?	En los repositories (AuthRepository, PatientRepository, CompanionRepository); las pantallas nunca hablan con Firebase.
¿Por qué no Room?	Firestore es la fuente de verdad y ya tiene caché offline; DataStore cubre preferencias. Room se añadiría si se necesitara historial local completo.
¿Cómo funciona sin conexión?	Firestore conserva la caché; los Flows muestran el último estado y se actualizan al reconectar.
¿Cómo se protege la privacidad?	Reglas de Firestore: cada uno escribe lo suyo, el acompañante solo lee el día, invitaciones de un solo uso y con expiración.
¿Qué pasa al cerrar sesión?	Se limpian las preferencias; una cuenta con correo se recupera al iniciar sesión; una anónima sin vincular se pierde y la UI lo advierte.
¿Cómo se conserva el estado al rotar?	ViewModel + StateFlow; rememberSaveable para estado de UI.
¿Por qué inyección manual?	App pequeña, dependencias explícitas y sustituibles en pruebas; Hilt añadiría complejidad innecesaria.
¿Qué falta?	Repositorio Git, release firmado y, para Play Store, borrado de cuenta y política de privacidad.

10.3 Mapa de código para la defensa

Tema	Archivos
Entrada	MainActivity.kt, MedsApp.kt
Autenticación	AuthRepository.kt, AppViewModel.kt, AuthScreen.kt
Estado del paciente	PatientViewModel.kt
Acceso a Firestore	PatientRepository.kt, CompanionRepository.kt, FirestoreMappers.kt
Navegación	MedsRoot.kt, PatientApp.kt
Persistencia local	PreferencesStore.kt
QR	QrCodeGenerator.kt, PairingCode.kt, PatientCompanionScreen.kt
Biometría	BiometricGate.kt
Widget	NextDoseWidgetProvider.kt, next_dose_widget_info.xml
Alarmas	AlarmScheduler.kt, MedicationAlarmReceiver.kt, DoseActionReceiver.kt
Seguridad	firestore.rules

11. Cobertura de la rúbrica

La rúbrica califica sobre 100 puntos y aporta el 60 % de la unidad: (puntos obtenidos / 100) × 60. Esta tabla resume dónde se cubre cada criterio y qué falta antes de la entrega.

Criterio	Máx.	Evidencia en este documento	Estado
1. Arquitectura MVVM y organización del código	20	Secciones 3 y 10.3: capas, StateFlow, repositorios, DI manual y recorridos.	Cubierto
2. Consumo de API y tratamiento de datos	15	Sección 4: Firebase Auth + Firestore, tiempo real, caché y errores. Matiz: no usa Retrofit.	Cubierto con matiz
3. Interfaz con Jetpack Compose y navegación	15	Sección 5: NavHost, rutas, estados, tema y componentes.	Cubierto
4. Gestión de datos y persistencia local	10	Sección 6: DataStore, caché de Firestore, SharedPreferences y alarmas.	Cubierto
5. Integraciones del dispositivo y generación de archivos	15	Sección 7: QR (generar/escanear), biometría y widget; alarmas como extra.	Cubierto
6. Estabilidad, validaciones y experiencia de uso	10	Sección 8: validaciones, errores, estados y pruebas en emulador y Xiaomi.	Cubierto
7. Repositorio, documentación y versión entregable	5	Sección 9: README, APK y 19 evidencias listos. Pendiente: repo Git y release firmado.	Parcial
8. Demostración práctica y defensa técnica	10	Sección 10: guion de 10 minutos, mapa de código y preguntas probables.	Cubierto
Total	100	Criterios 1, 3, 4, 5, 6 y 8 cubiertos; 2 con matiz; 7 parcial.	

11.1 Riesgos y acciones

Riesgo	Nivel	Acción
La cátedra exige Retrofit/REST y no acepta Firebase	Alto	Añadir una API REST (openFDA) o defender el SDK de Firebase como cliente equivalente; documentar el flujo.
Reglas de Firestore sin publicar	Alto	Publicar firestore.rules y probar escrituras en producción.
Repositorio Git y ramas	Medio	Inicializar main/develop/feature y subir el proyecto.
APK debug	Medio	Configurar keystore y generar release firmado.
Cuenta anónima sin vincular	Bajo	Vincular un correo en la demo.

12. Anexos

A. Comandos útiles

```
# Compilar la variante debug
.\gradlew.bat :app:assembleDebug

# Instalar en un dispositivo o emulador conectado
adb install -r app\build\outputs\apk\debug\app-debug.apk

# Emuladores de Firebase (opcional, para pruebas locales)
npx --yes firebase-tools@latest emulators:start --only auth,firestore --project medsproto
adb reverse tcp:9099 tcp:9099
adb reverse tcp:8080 tcp:8080

# Publicar reglas (requiere Firebase CLI)
firebase deploy --only firestore:rules --project medsproto
```

B. Cuentas de prueba

Para la demostración se crearon cuentas de prueba en el proyecto `medsproto`: `prueba.meds.ian@example.com`, `prueba.meds.ian2@example.com` y `prueba.meds.ian3@example.com`. La contraseña se comparte por un canal aparte del repositorio. Son cuentas de ejemplo: deben eliminarse antes de publicar la app.

C. Glosario

Término	Significado
MVVM	Model-View-ViewModel: separa datos, estado e interfaz.
StateFlow	Flujo de estado observable e inmutable que consume Compose.
Repository	Capa que encapsula el acceso a datos y las reglas de negocio.
Batch	Escritura atómica de varios documentos en Firestore.
BaaS	Backend as a Service: Firebase ofrece auth y base de datos sin servidor propio.
Nudge	Recordatorio breve que el acompañante envía al paciente.
QR	Código bidimensional que aquí transporta una invitación temporal.

D. Checklist final

- Publicar las reglas de Firestore y probar un alta de medicamento.
- Vincular un correo a la cuenta de demostración para conservar los datos.
- Inicializar el repositorio Git con main, develop y feature; subir el proyecto.
- Firmar el release y guardar el APK/AAB junto a la entrega.
- Revisar README, evidencias y este documento antes de la exposición.
- Ensayar el guion de 10 minutos y las preguntas de la sección 10.2.

Documento de trabajo del equipo MEDs. Versión 1.0 · Noviembre 2026.